

Introduction to Transformers: Decoding and Pretraining

CS60216: Safety Fundamentals for Generative AI

Jan 12th, 2026

Autoregressive nature

$\square \rightarrow o/p$

Decoder

highest prob

prob dist

over
the entire
vocab

reference

FFN

cross-attention

(masked) self-attention

Layers

$\pm d$
embedd_{ing}

$\square \quad \square \quad \square \quad \square$

Which words do we generate at each step?

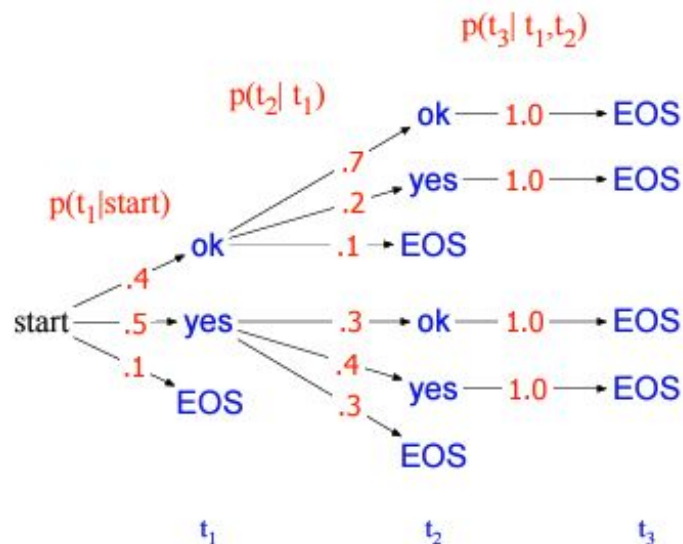
Greedy Decoding

One simple way is to always generate the most likely word given the context

Generating the most likely word given the context is called *greedy decoding*.

$$\hat{w}_t = \arg \max_{w \in V} P(w | w_{<t})$$

What is the issue with greedy decoding?



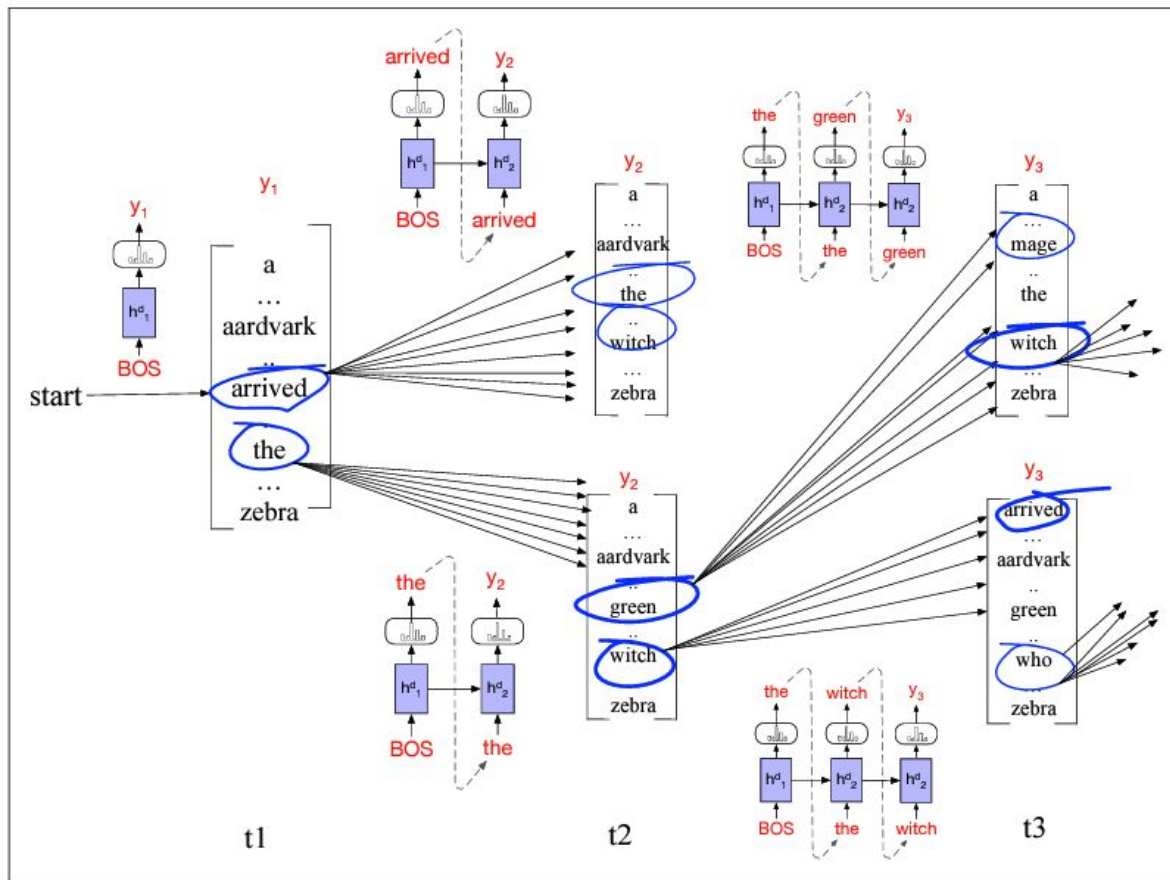
- *What will greedy decoding choose?*
- *What is globally optimal?*

Beam Search

Core Idea

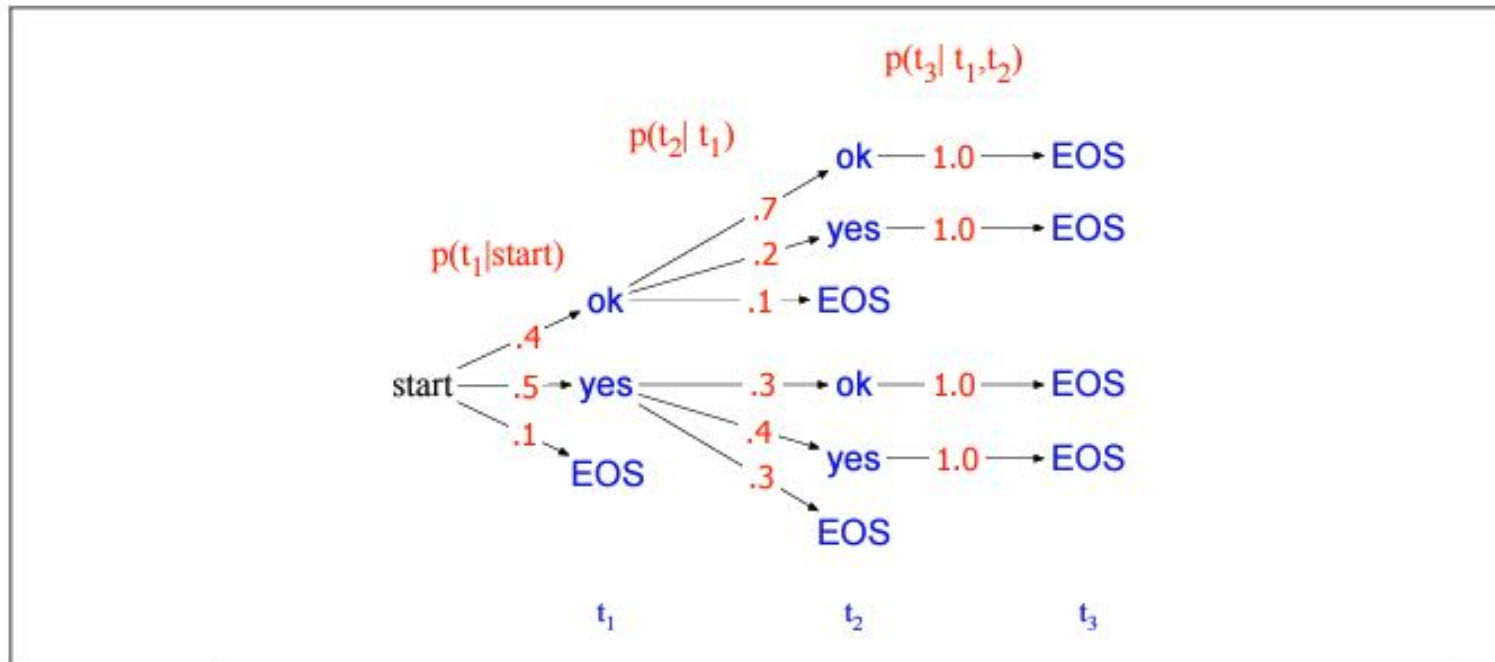
- On each step of the decoder, keep track of the k most probable partial translations (hypotheses)
- k is the beam size / beam width (in practice around 5 to 10)

Beam Search: How does it work?



<https://web.stanford.edu/~jurafsky/slp>
3

Beam Search on the prev. example



Take Beam Size = 2. Is the Beam search solution same as the optimal solution?

Greedy and Beam Search are deterministic!

- Greedy decoding as well as Beam Search decoding will give a “deterministic” output
- Other common decoding algorithms involve “sampling”, and bring in some degree of “randomness”

$x \sim p(x) \rightarrow$ choose x by sampling from the distribution $p(x)$

Random Sampling

```
i ← 1
wi ∼ p(w)
while wi ≠ EOS
  i ← i + 1
  wi ∼ p(wi | w<i)
```

Quality vs Diversity trade-off

Various sampling methods enable trading off two important factors in generation: *quality* and *diversity*.

Quality vs Diversity trade-off

- Methods that emphasize the most probable words tend to produce more coherent and accurate generations but also tend to be repetitive and boring
- Methods that give bit more weight to the middle probability words tend to be more creative and diverse, but likely to be incoherent and less factual

Random Sampling with Temperature

Intuition from thermodynamics

A system at a *high temperature* is flexible and can explore various states, while a system at a *low temperature* is likely to explore a subset of lower energy (better) states

How is this implemented

Divide the logits by a temperature parameter $\tau \in (0, 1]$ before passing it through softmax

Random sampling: $y = \text{softmax}(u)$

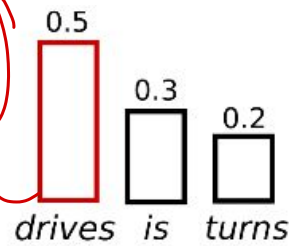
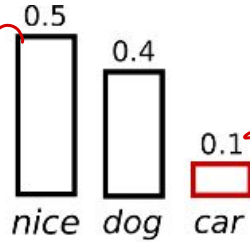
Random sampling with temperature: $y = \text{softmax}(\underline{u/\tau})$

Why does this work?

$$\frac{u}{\tau}$$

$$\tau \sim \begin{matrix} 0.1 \\ 0.2 \end{matrix}$$

random sample

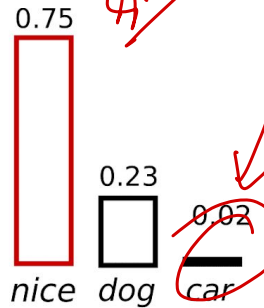


Without scaling, random sampling often generates incoherent gibberish

The

car

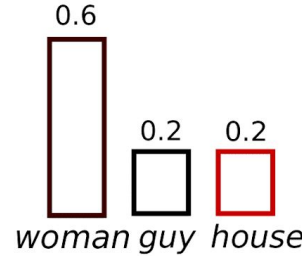
drives



sharpens

$$p(y_i | x) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Higher T: softens probabilities.
Lower T: sharpens probabilities.



The

nice

house

Top-k Sampling

In Top-k Sampling, only Top k tokens (as per prob.) are considered for generation, so the less probable words would not have any chance

1. Choose in advance a number of words k
2. For each word in the vocabulary V , use the language model to compute the likelihood of this word given the context $p(w_t | \mathbf{w}_{<t})$
3. Sort the words by their likelihood, and throw away any word that is not one of the top k most probable words. 
4. Renormalize the scores of the k words to be a legitimate probability distribution.  
5. Randomly sample a word from within these remaining k most-probable words according to its probability.

Nucleus Sampling or Top-p sampling

Issues with Top-k Sampling

Shape of the probability distribution differs in different contexts. Top-k may include most of the probability mass in some cases, and very small mass in other cases.

Nucleus Sampling or top-p sampling

Keep not the top k words but top p percent of the probability mass

Given a distribution $P(w_t|w_{<t})$, top-p vocabulary $V^{(p)}$ is the smallest set of words such that

$$\sum_{w \in V^{(p)}} P(w|w_{<t}) \geq p$$

Practice problem

Suppose you have a vocabulary of size 5 and during decoding, the output vector is $[3, -1, 2, 1, -2]$. Write down the effective probability distribution when you use the following sampling strategies.

- Random sampling with temperature 0.5
- Top-2 sampling
- Nucleus sampling with $p = 0.5$

Try this problem

Consider the following probability distributions for your vocabulary tokens in decreasing order of probabilities:

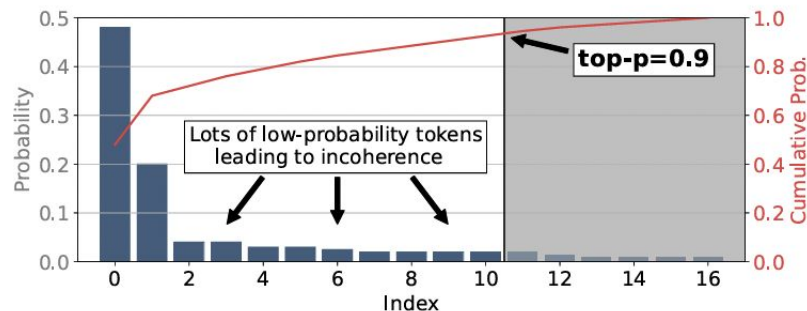
$\{0.5, 0.2, 0.02, 0.02, 0.02, 0.02, 0.02, 0.01, \dots\}$

$\{0.15, 0.13, 0.12, 0.1, 0.09, 0.08, 0.07, 0.06, 0.01, \dots\}$

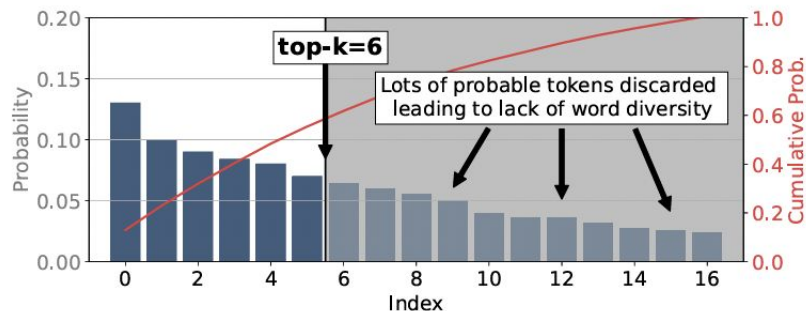
Suppose that you use nucleus sampling with $p = 0.8$. Will it be a good choice for both the cases? Can you think of any other threshold for Top-p / Top-k that will be good? Briefly justify your answer.

Can you think of another sampling method that works better than these two methods for both the cases? Describe your strategy along with a choice of hyper-parameter that works for both these cases.

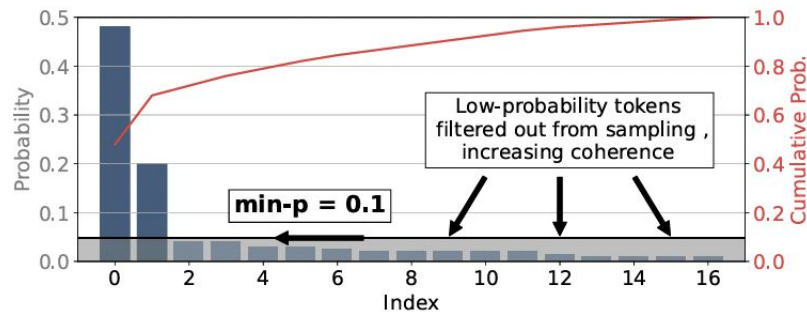
Min-p Sampling (ICLR'25)



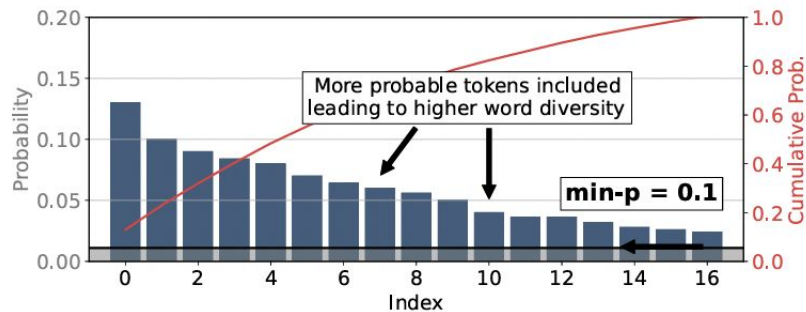
(a)



(b)



(c)



(d)

Min-p Sampling (ICLR'25)

1. **Calculate the Maximum Probability:** Identify the maximum probability token in the distribution, denoted as $p_{\max} = \max_{v \in \mathcal{V}} P(v \mid x_{1:t-1})$.
2. **Truncation Threshold:** Set a base probability threshold, $p_{\text{base}} \in (0, 1]$ Scale it by $p_{\max}$:

$$p_{\text{scaled}} = p_{\text{base}} \times p_{\max} \quad (1)$$

This threshold ensures that tokens with sufficiently high relative probabilities are considered while filtering out less probable tokens in a context-dependent manner.

3. **Define Sampling Pool:** Construct pool $\mathcal{V}_{\min}$ of tokens with probabilities $\geq p_{\text{scaled}}$:

$$\mathcal{V}_{\min} = \{v \in \mathcal{V} : P(v \mid x_{1:t-1}) \geq p_{\text{scaled}}\} \quad (2)$$

4. **Sample from Pool:** Sample the next token x_t from the normalized set $\mathcal{V}_{\min}$:

$$P'(v) = \frac{P(v \mid x_{1:t-1})}{\sum_{v' \in \mathcal{V}_{\min}} P(v' \mid x_{1:t-1})} \quad \text{for } v \in \mathcal{V}_{\min} \quad (3)$$

Any issue with *min-p* sampling?

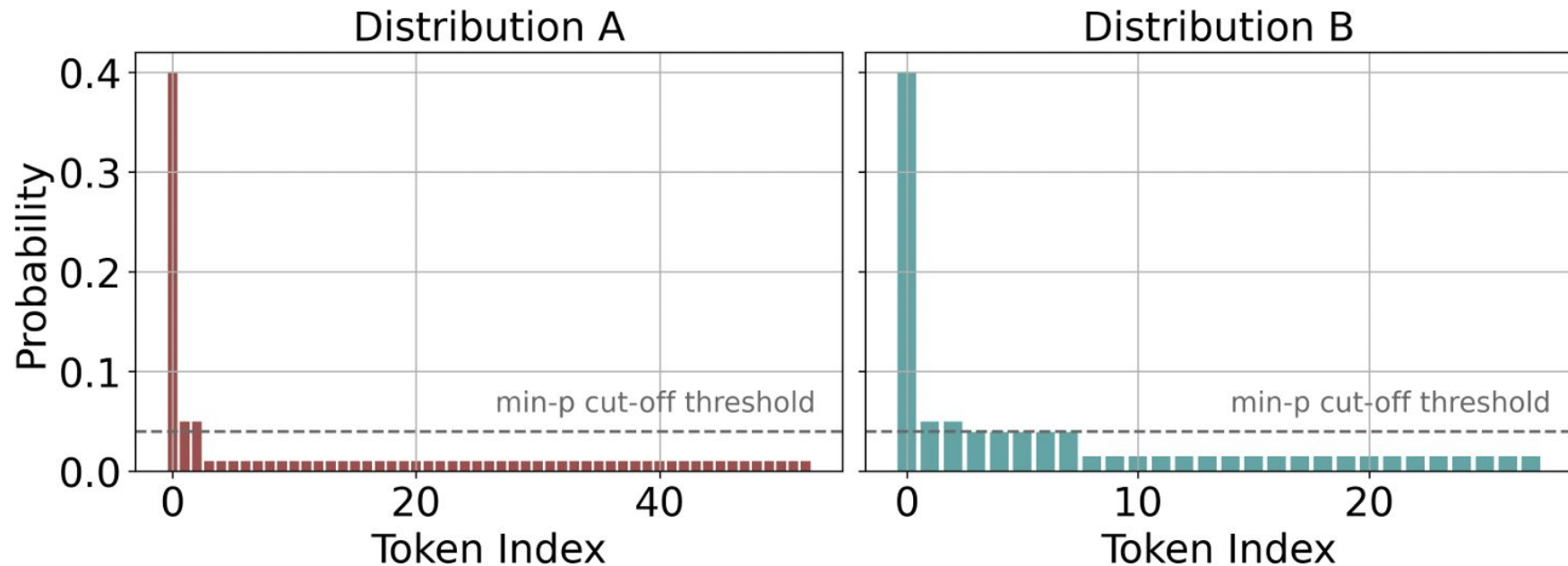


Figure 1: Probability distribution of two different types with associated *min-p* threshold.

Top-H decoding (Neurips'25)

Algorithm 1 Top-H: proposed greedy token selection algorithm

Require: Probability mass function $p = (p_1, p_2, \dots, p_n)$, entropy threshold coefficient $\alpha \in (0, 1)$

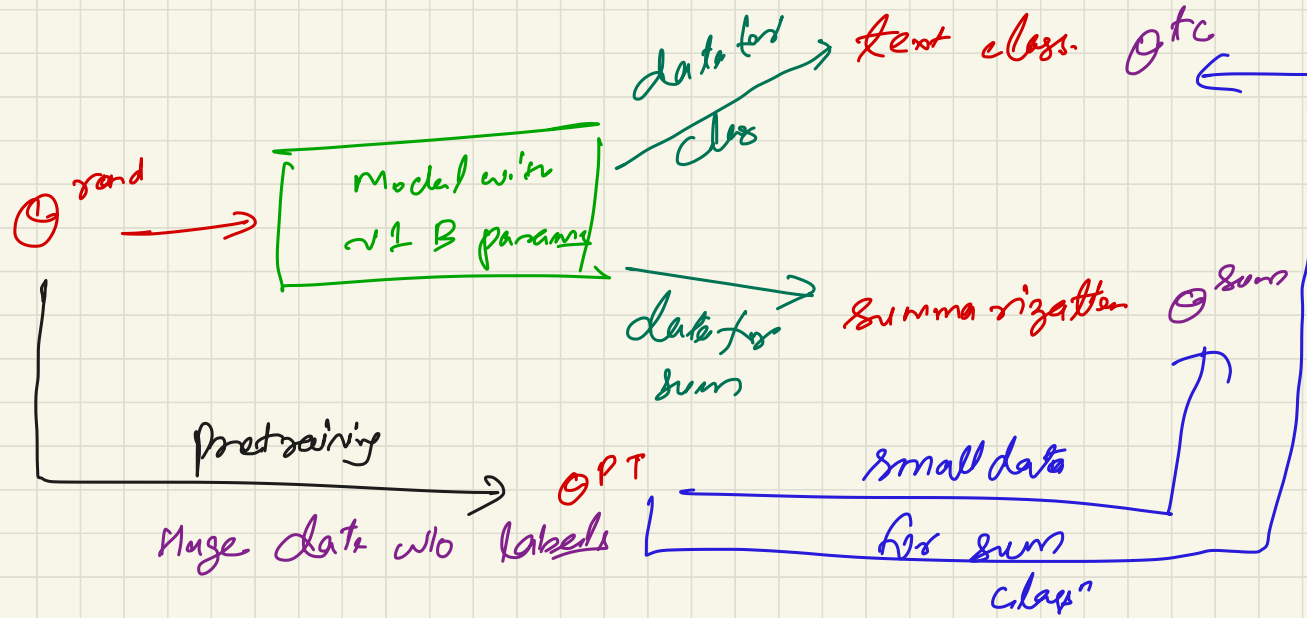
Ensure: Selected token set S

- 1: Sort tokens in descending order of probability: $p_1 \geq p_2 \geq \dots \geq p_n$
 - 2: Initialize $S \leftarrow \emptyset, H(q) \leftarrow 0$
 - 3: **for** each token i in sorted order **do**
 - 4: Add token i to S
 - 5: Compute updated distribution q over S
 - 6: Compute entropy $H(q)$
 - 7: **if** $H(q) > \alpha \cdot H(p)$ **then**
 - 8: Remove token i from S
 - 9: **break**
 - 10: **end if**
 - 11: **end for**
 - 12: **return** S
-

greedy, beam } $\Rightarrow$ deterministic

Random Sampling, Top-k, Top-p, Min-p, Top-H } induce
↓ randomness,
diversity
with temp.

Pretrained Transformers



Transfer Learning with Language Modeling

Language modelling is the popular choice of pretraining in NLP.

The key idea is to **train a neural network language model (on vast text corpora)**. And **then, use it to transfer that knowledge to any target task in NLP** we care about.

This is a **very successful** idea in NLP:

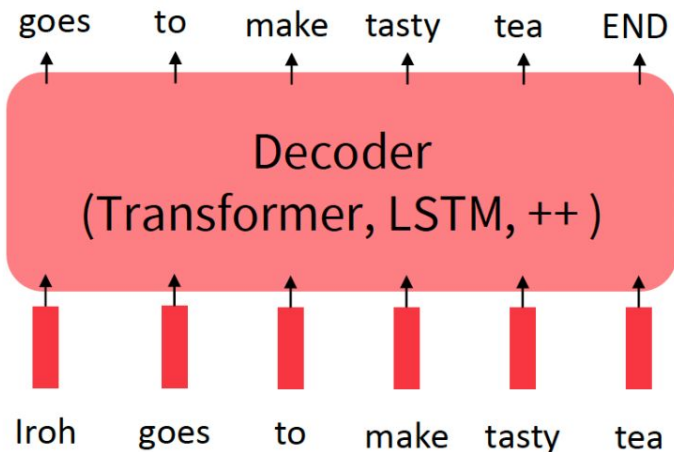
- Supervised systems for various NLP tasks used to require millions of examples to learn tasks. NLP systems that use language models as a starting point, need much fewer - thousands of examples to do so!

The Pretraining / Finetuning paradigm

Pretraining can improve NLP applications by serving as parameter initialization.

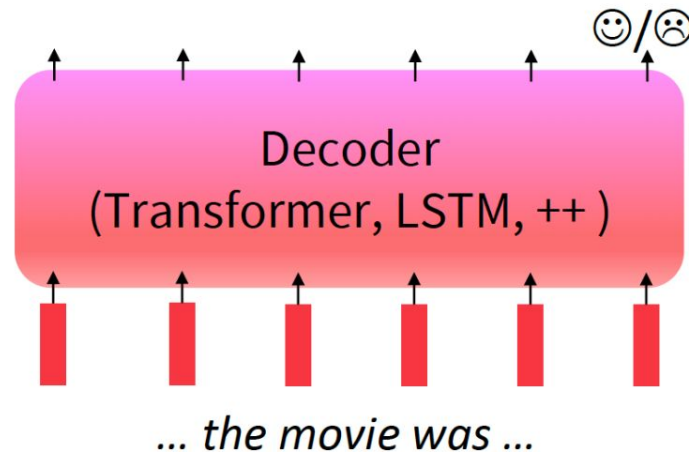
Step 1: Pretrain (on language modeling)

Lots of text; learn general things!

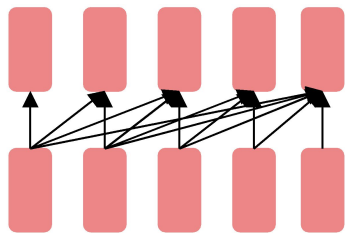


Step 2: Finetune (on your task)

Not many labels; adapt to the task!



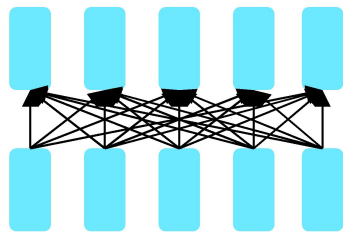
Three architectures for large language models



Decoders ✓

GPT, Claude,
Llama, Mixtral

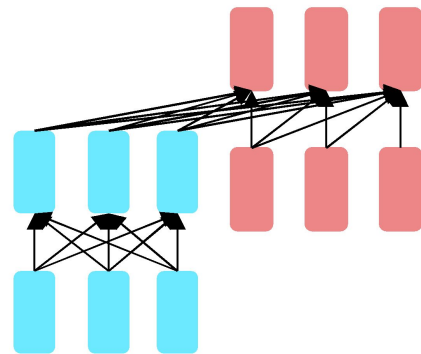
*next word
prediction*



Encoders ✓

BERT family

*Mask Language Models
(MLM)*



Encoder-decoders ✓

Flan-T5 ✓

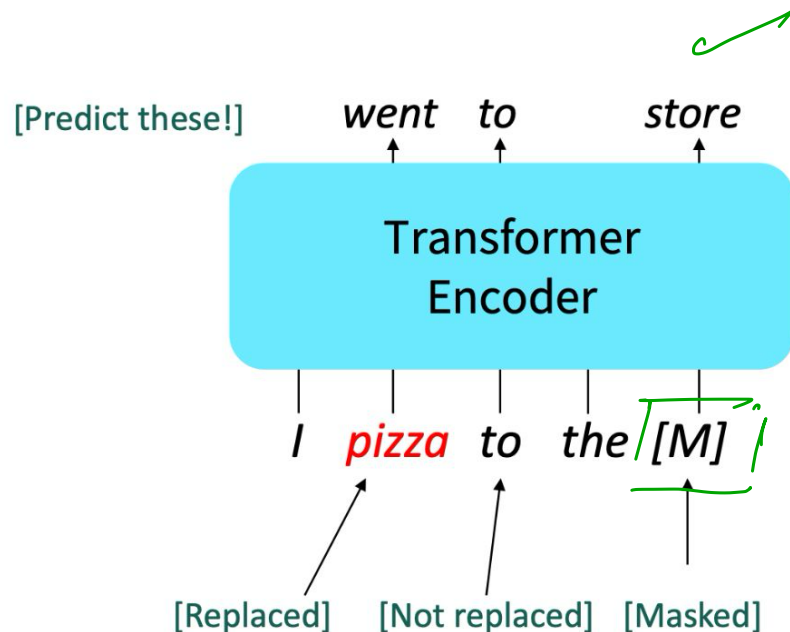
BART

BERT: Bidirectional Encoder Representations from Transformers

Devlin et al., 2018 proposed the “Masked LM” objective and **released the weights of a pretrained Transformer**, a model they labeled BERT.

Some more details about Masked LM for BERT:

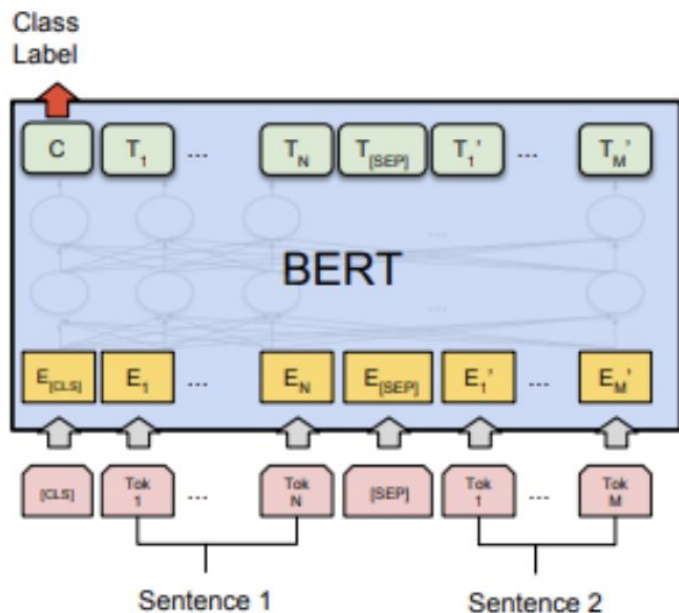
- Predict a random 15% of (sub)word tokens.
 - Replace input word with [MASK] 80% of the time
 - Replace input word with a random token 10% of the time
 - Leave input word unchanged 10% of the time (but still predict it!)
- Why? Doesn't let the model get complacent and not build strong representations of non-masked words. (No masks are seen at fine-tuning time!)



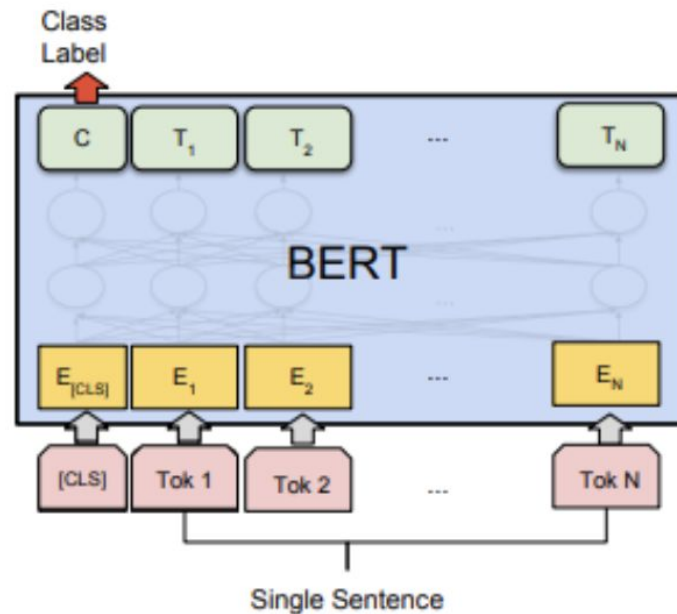
More Details about BERT

- Two models were released:
 - BERT-base: 12 layers, 768-dim hidden states, 12 attention heads, 110 million params.
 - BERT-large: 24 layers, 1024-dim hidden states, 16 attention heads, 340 million params.
- Trained on:
 - BooksCorpus (800 million words)
 - English Wikipedia (2,500 million words)
- Pretraining is expensive and impractical on a single GPU.
 - BERT was pretrained with 64 TPU chips for a total of 4 days.
 - (TPUs are special tensor operation acceleration hardware)
- Finetuning is practical and common on a single GPU
 - “Pretrain once, finetune many times.”

Using BERT for different tasks



(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(b) Single Sentence Classification Tasks:
SST-2, CoLA

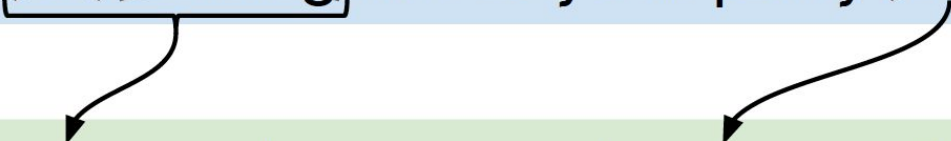
T5: A New Training Objective

Span Corruption

decode out the spans that were corrupted

Original text

Thank you ~~for inviting~~ me to your party ~~last~~ week.



Inputs

Thank you <X> me to your party <Y> week.

Targets

<X> for inviting <Y> last <Z>

T5 can be used for various tasks

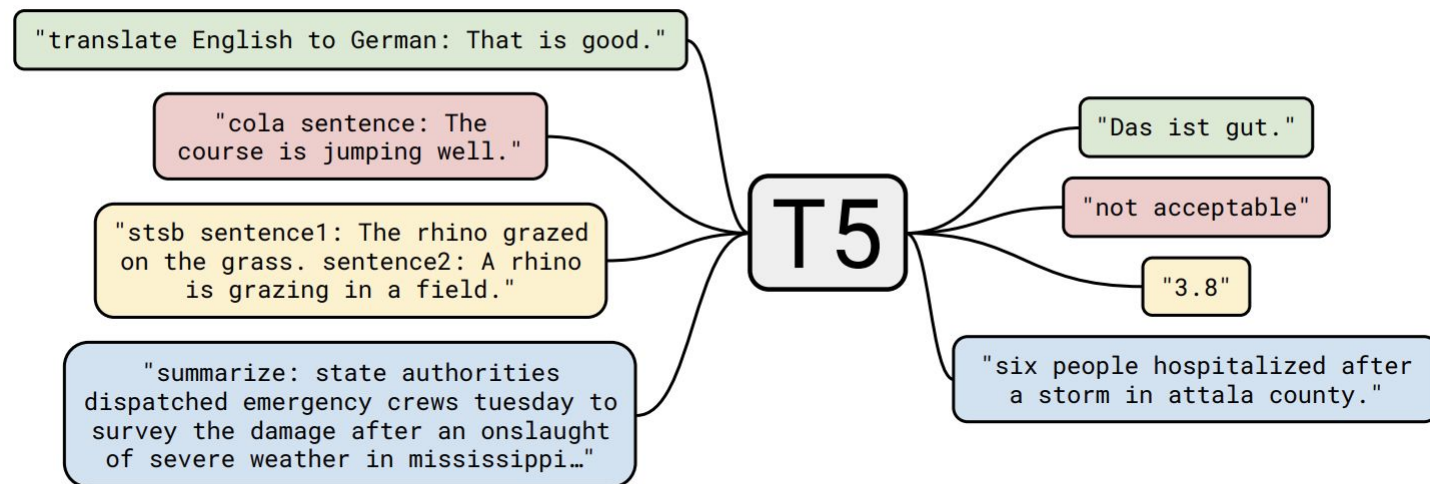


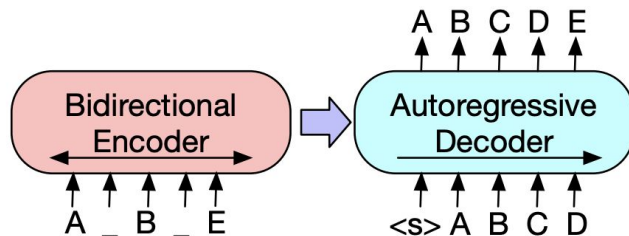
Figure 1: A diagram of our text-to-text framework. Every task we consider—including translation, question answering, and classification—is cast as feeding our model text as input and training it to generate some target text. This allows us to use the same model, loss function, hyperparameters, etc. across our diverse set of tasks. It also provides a standard testbed for the methods included in our empirical survey. “T5” refers to our model, which we dub the “**T**ext-**t**o-**T**ext **T**ransfer **T**ransformer”.

Bi-directional and Auto-Regressive Transformers (BART)



(a) BERT: Random tokens are replaced with masks, and the document is encoded bidirectionally. Missing tokens are predicted independently, so BERT cannot easily be used for generation.

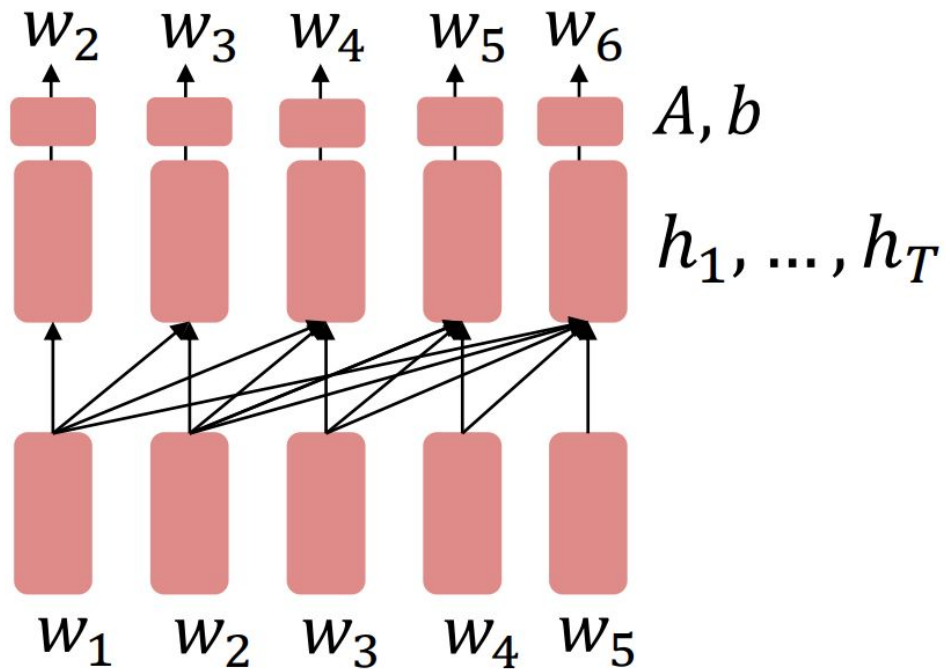
(b) GPT: Tokens are predicted auto-regressively, meaning GPT can be used for generation. However words can only condition on leftward context, so it cannot learn bidirectional interactions.



(c) BART: Inputs to the encoder need not be aligned with decoder outputs, allowing arbitrary noise transformations. Here, a document has been corrupted by replacing spans of text with mask symbols. The corrupted document (left) is encoded with a bidirectional model, and then the likelihood of the original document (right) is calculated with an autoregressive decoder. For fine-tuning, an uncorrupted document is input to both the encoder and decoder, and we use representations from the final hidden state of the decoder.

Pretraining Decoders

It's natural to pretrain decoders as language models



Generative Pretrained Transformer (GPT)

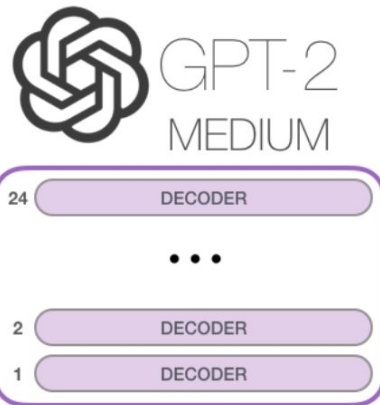
- Transformer decoder with 12 layers, 117M parameters.
- 768-dimensional hidden states, 3072-dimensional feed-forward hidden layers.
- Byte-pair encoding with 40,000 merges
- Trained on BooksCorpus: over 7000 unique books.
- Contains long spans of contiguous text, for learning long-distance dependencies.

Vocabulary larger than BERT

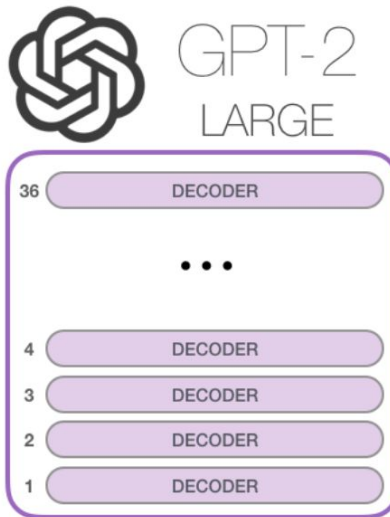
Pretrained Decoders



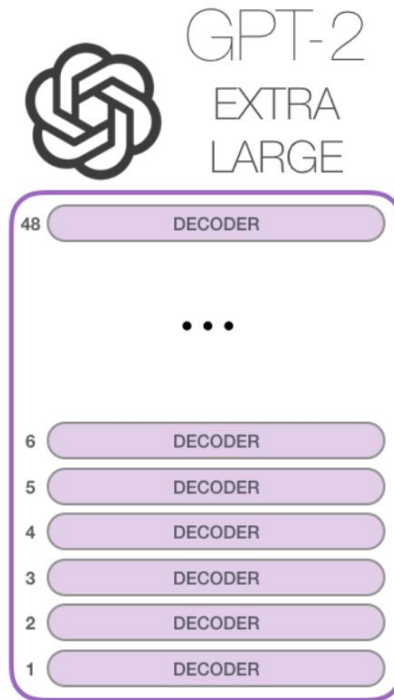
Model Dimensionality: 768



Model Dimensionality: 1024



Model Dimensionality: 1280



Model Dimensionality: 1600

GPT-2: Beyond Language Modeling

Basic Idea

The decoder can work with a prompt!

Any NLP task can be expressed in a probabilistic framework as estimating a conditional distribution $p(\textit{output}|\textit{input})$.

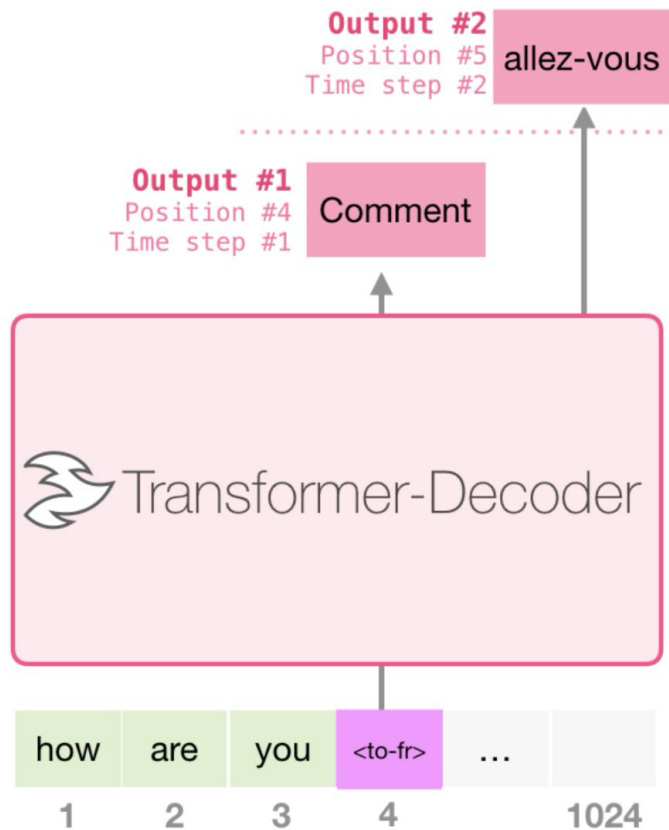
For instance, reading comprehension training example

(answer the question, document, question, answer)

GPT-2: Machine Translation

Training Dataset

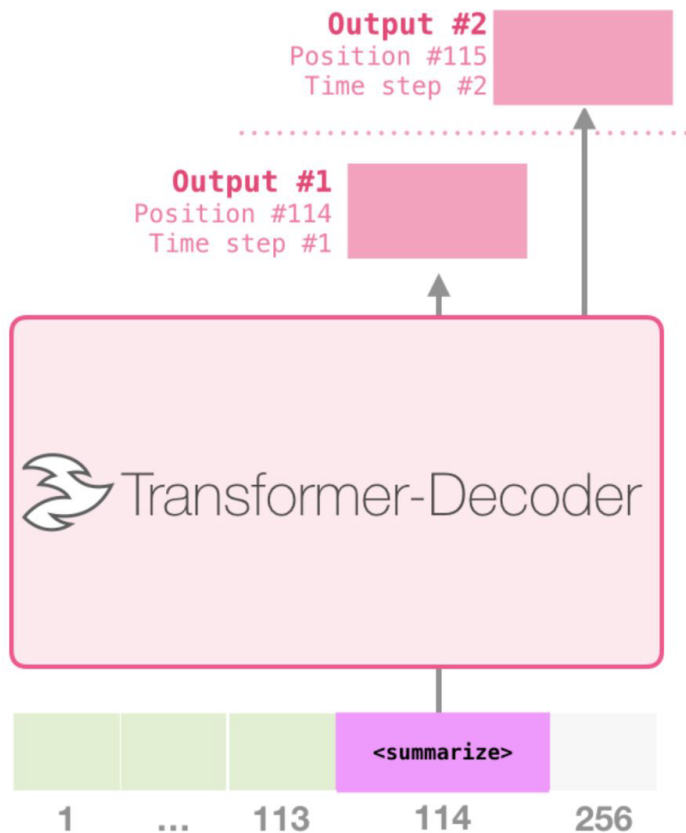
I	am	a	student	<to-fr>	je	suis	étudiant
let	them	eat	cake	<to-fr>	Qu'ils	mangent	de
good	morning	<to-fr>	Bonjour				



GPT-2: Summarization

Training Dataset

Article #1 tokens	<summarize>	Article #1 Summary
Article #2 tokens	<summarize>	Article #2 Summary
padding		
Article #3 tokens	<summarize>	Article #3 Summary
padding		



How to format inputs?

Natural Language Inference

Premise: *The man is in the doorway*
Hypothesis: *The person is near the door* } **entailment**

Radford et al., 2018 evaluate on natural language inference.

Here's roughly how the input was formatted, as a sequence of tokens for the decoder.

[START] *The man is in the doorway* [DELIM] *The person is near the door* [EXTRACT]

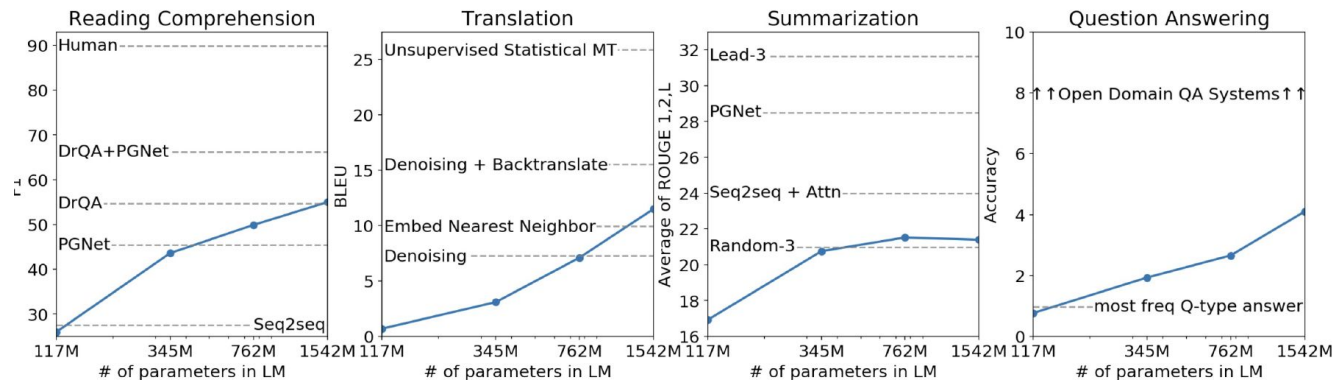
The linear classifier is applied to the representation of the [EXTRACT] token.

GPT Series

GPT-2 (Radford et al. 2019) scaled the GPT model to a larger model.

Demonstrated that language models are able to do NLP tasks **without any explicit supervision (zero-shot)**.

- e.g., when conditioned on a document plus questions, the answers generated by GPT-2 reach 55 F1 on the CoQA dataset.



Zero-shot task performance of GPT-2 as a function of model size on many NLP tasks

GPT Series

Generative Pretrained Transformer series of models from OpenAI

GPT-3 (Brown et al. 2020) further scaled the GPT-2 model to 175 Billion (100 times larger compared to the largest GPT-2 model).

In addition to improved zero-shot performance, GPT-3 also exhibits strong few-shot “**in-context learning**” ability

The three settings we explore for **in-context learning**

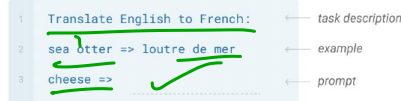
Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.



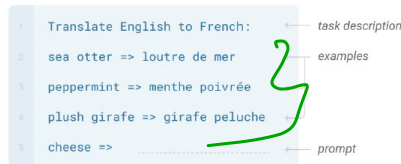
One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.



Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.



Traditional fine-tuning

Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.



In in-context learning, we append the task description and a few demonstrations of the task to the original input as the input for the language model, which produces the prediction by auto-regressively predicting next tokens

Comparison of GPT series

Characteristic	GPT-1	GPT-2	GPT-3
Parameters	117 Million	<u>1.5 Billion</u>	175 Billion ✓
Decoder Layers	12	48	96
Context Token Size	512	1024	2048
Hidden Layer Size (model dim)	<u>768</u>	<u>1600</u>	<u>12288</u>
Batch Size	64	512	3.2M

Image Source: “Large Language Models: A Deep Dive”